Real-Time Football Match Tracking and Alert System
COMP 208 Requirement Analysis (CA2)

Team Members:

Hamza Al Zudi Garcia-Olalla	Balqis Binti Abdul Halim	Best Boonthanomwong	Ziad Azmi
Jacob Daya	Aya El Khayat	Tom Sutton	

1. Project Description

Football fans are increasingly using digital platforms to follow live matches when watching in real-time isn't practical. Most existing applications provide live scores and basic statistics, but often lack personalization, contextual analytics, or timely alerts tailored to users.

This project proposes a real-time football match-tracking and alert system that provides live updates on scores, possession shifts, disciplinary events (fouls, yellow and red cards), and advanced metrics such as expected goals (xG). Users can select specific teams or matches to follow and receive notifications for significant events.

The system will pull live data from API-Football, presenting it through a user-friendly interface for both casual fans and those seeking deeper insights. While many platforms offer live updates, they typically focus on generic reporting with limited explanations of match dynamics. This project aims to differentiate itself by combining personalized alerts with clear, interpretable analytics, promoting clarity and informed engagement.

2. Aim and Objectives

2.1 Aims

- To design and develop a real-time football match tracking system that provides accurate and timely live updates.
- To enhance user engagement through personalised notifications and contextual football analytics.
- To apply software engineering principles in the design of scalable, data-driven application.

2.2 Objectives

- Retrieve live football match data from an external football data API (API-Football).
- Process key in-game events such as goals, fouls, substitutions, and cards with time-stamped entries.
- Display both traditional match statistics and selected advanced metrics such as expected goals (xG), where available.
- Allow users to select favourite teams and matches.
- Implement a configurable notification mechanism for significant match events and provide a user-friendly interface optimised for live match interaction.
- Design the system with scalability and extensibility in mind.
- Prioritise core real-time tracking and alert functionality, with advanced analytics treated as extendable objectives if constraints arise.
- Allow the comparison of different players and teams.

3. Key Literature and Background Reading

Football Data & Live Sports Systems

Football APIs provide footballing data that are typically structured into several components such as fixtures, match events, and match statistics. Fixtures define the context of a match, including teams, competition, kick-off time and unique match identifiers. Match events represent in-game actions such as goals, substitutions and disciplinary

actions and are recorded as time-stamped entries that allow applications to construct real-time event timelines. Match statistics consist of aggregated numerical data summarizing team performance.

However, there are several challenges that are presented to live football data systems when showing live updates, such as reliability and consistent data. One challenge is that because of data being collected and then verified, it can lead to short delays as updates are not being sent instantly and instead, after a time lag. However, previous documentation on live data systems can suggest that using event-based models, where data is updated only when a significant match event occurs, instead of continuously updating the system, will reduce unnecessary data requests and hence, improve overall system efficiency.

Currently, there are existing live football score applications, such as FotMob, that commonly provide live match scores, updates, statistics, selected team information, and enabled notifications for special events (FotMob, n.d.).

However, some references highlight a few technical drawbacks in existing live score systems like FotMob. Typical constraints include handling large volumes of live data, API rate limits, and maintaining the quality of the app long-term, as APIs are constantly updated over time, which can lead to program changes (Jaja, 2025). Poor integration practices and lack of monitoring of API changes can lead to a system delay or even failure during periods of high traffic. This project addresses these issues by focusing on efficient event-based data handling alongside a simpler interface which can improve reliability and reduce heavy data processing.

4. Development and Implementation Summary

To implement this design, a client-server architecture will be used. Backend logic will be written in Python, which is chosen to make API calls and manage the database due to the variety of libraries such as *requests* and *sqlite3*. The database will be written in SQL, and it will be used to store information such as players, teams and leagues. For the frontend, HTML templates will be created and designed with CSS to maintain design consistency between pages.

For data retrieval, a caching system will be used to minimise API calls and to reduce latency. When a request is made to a database, the program will check if the data requested has been cached inside the database and return that information. If the data is absent, then the API will be called. Since historical match data is mostly static (unless in events such as VAR corrections), this data can be used for future requests. However, the request may not exist. To limit API calls, the failure will also be cached.

To deliver information to the client, dynamic routing will be necessary. Specific HTML templates will be created for key entities in the table, such as players, teams and matches. When a specific URL is requested, the server will first check if the page exists. If not, then the data retrieval process above will be executed. With the data returned, the server will build up the page from the template.

The top-down development process will be used, starting with the database schemas and the insertion of information through the API. This will ensure that backend logic is stable and delivers data in the right format. When this is done, frontend development can begin, starting with dynamic routing which can ensure that information is displayed according to the server. Git will also be preferred to track changes to the code.

5. Data Sources

To get the most accurate data possible within the constraints of the project, we will have to use multiple sources. First, API-football (API-Football, 2025) provides an interface for querying most of the needed data (e.g. match scores, upcoming events, and player statistics). However, to be able to incorporate advanced statistics like xG or xA, we will also be using S*tatsBomb Open Data* (Statsbomb, n.d.) which is a set of data released by StatsBomb for certain leagues, which includes more detailed statistics - each match is split into a list of events that occurred during the match and events are split into different categories (e.g. pass, shot, ball intercept, duel, recovery etc). The S*tatsBomb Open Data* is completely public, simply asking to be listed as a data source when distributing any analysis based on their data.

Another potential data source is *understat* (amosbastian, 2025) which while not providing completely detailed data that StatsBomb has, it does have up-to-date xG statistics that StatsBomb lacks and all their data is free to use for non-commercial uses.

All user passwords will be hashed using an industry-standard KDF (Either bcrypt or Argon2id) and stored in an SQL database. This will ensure that all user data is stored in the most secure way possible.

6. Testing and Evaluation

To ensure the quality and durability of the final application, we will be applying a series of tests and evaluating user feedback before finalising the project.

The tests we will apply are listed as follows:

- When a football match is being played, real-time data should be shown after X seconds. (With X to be decided later in development, when we have a better understanding of the limitations of our system)
- When a user begins watching a football match and some stat changes (e.g. a goal is scored) simultaneously, the application should catch the discrepancy between the user-side stats (the stats that the user is being shown) and the server-side stats. (the actual stats within the database)

The methods we will use to determine our project's final requirements are listed as follows:

- A questionnaire, covering the performance and quality of all key features, will be utilised to find how satisfied the user is with different features of the website. This will also allow us to compare the user satisfaction of different demographics. (e.g. casual fans and experts)
- Throughout production, we will create small lists of requirements for each module before we implement it (e.g. before implementing a home page, we would create a small list of requirements that may include being able to navigate from the home page to some other page in 1 or 2 clicks).

  The number of requirements fulfilled will be considered when evaluating the entire project.

7. Ethical Considerations

This study will be conducted following the ethical guidelines set by the University of Liverpool, with full agreement from all members of the research team. We are committed to upholding academic integrity by properly citing all sources to avoid plagiarism.

The system will utilize third-party football data from API-Football and StatsBomb transparently and lawfully. API-Football provides football data through an API that requires registration and an API key. The data we use will comply with the provider's terms and conditions, and the raw data will not be redistributed. Additionally, StatsBomb Open Data is a publicly available dataset provided by StatsBomb for research and analysis purposes, making it suitable for this study. We will acknowledge StatsBomb as the source for all related data used.

Since this research involves creating a web application, safeguarding user information is a crucial factor. Users will only need to share minimal details, including a username and email address. Passwords will be securely hashed utilizing industry-standard key derivation methods such as bcrypt or Argon2id prior to being saved in an SQL database. The project will comply with GDPR guidelines. Users will have the ability to delete their accounts, and any related personal information will be removed. All data will be stored securely with password protection and will only be accessible to members of the research team.

During the testing phase, a voluntary questionnaire will be distributed to gather feedback on system performance and usability. Participants can withdraw at any time before submission. Responses will be anonymized and used only for academic evaluation. All data will be securely stored and deleted after the project ends. The study will be conducted fairly and without bias, with findings unaffected by external interests or coercion.

8. BCS Project Criteria

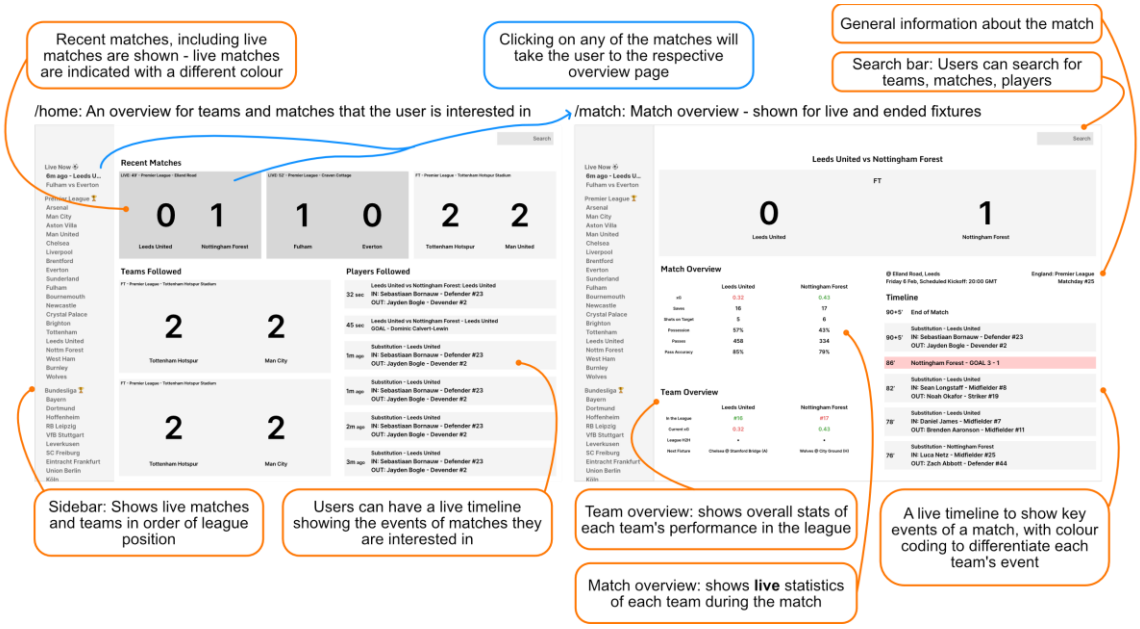
The project applies practical and analytical skills gained during the degree programme through different areas, such as applying analytical techniques to define functional and non-functional requirements. Object-oriented design techniques are applied to model core system entities and to define different responsibilities within the system. In addition, data modelling techniques are used to design an efficient data structure capable of supporting the real-time alert feature.

The project addresses a real-world need for football fans who are unable to watch matches live, whether it's in person or through a streaming service. For example, the '3 pm blackout' rule (Chowdhury, 2025) prevents football matches from being shown between 14:45 and 17:15 on Saturdays, which applies to the top two divisions, may stop a fan from being able to watch the match. By having a real-time match tracking and alert systems football fans can stay informed about key events of the match despite the viewing restrictions.

Self-management is also demonstrated through structured planning, where a Gantt chart will be used to plan tasks, milestones and to manage the upcoming deadlines of this project. The chart provides a breakdown of tasks making it more manageable and to make sure such tasks are completed in a timely manner.

Critical self-evaluation is incorporated within the project. While the Gantt chart helps to organize the workflow, adjustments and frequent revisions are required due to the changes in task complexity and any time constraints that may occur. This highlights the importance of flexibility within the workflow.

9. UI/UX Mock Up

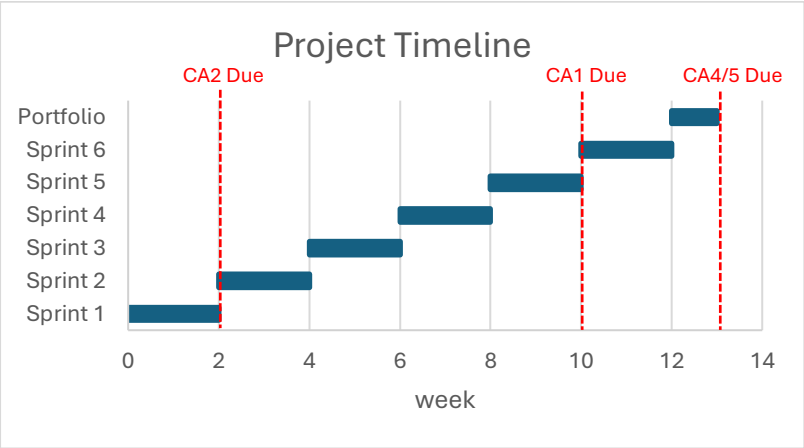


10. Project Plan, Risks, and Contingencies

10.1 Project Timeline 12-week development timeline (February 2 – May 11, 2026)

Sprint 1 (Weeks 1-2): CA2 Requirement Analysis submission (February 16), Figma mock ups, SQL database schema design, API provider evaluation (API-Football vs alternatives), GitHub repository setup with CI/CD.
Sprint 2 (Week 3-4): API integration, Database implementation, Caching system, Unit tests for backend.
Sprint 3 (Week 5-6): Event processing, Notification engine, HTML, Integration tests.
Sprint 4 (Week 7-8) Frontend UI completion, User accounts, Advanced metrics (xG), Performance testing.
Sprint 5 (Week 9-10): Testing, Questionnaire distribution, Bug fixes, Performance optimization, Video demo preparation.
Sprint 6 (Week 11): CA1 Demo submission (Apr 20), User manuals; Q&A preparation, System deployment.

Portfolio (Week 12-13): Design documentation (CA4), Final testing, Portfolio report, Peer assessment, CA5 submission (May 11)



10.2 Risks and Contingencies

Risk	Contingencies	Likelihood	Impact
API rate limit exceeded (100/day)	Aggressive caching, reduced polling frequency	Medium	High
Data breach/password leak	Bcrypt hashing (12 rounds), environment variables	Low	Very High - legal/ethical violation
API downtime	Caching to provide historical data	Low	Medium - Live data may not be able to be presented to the user
Denial of Service Attack	Implement DDoS protection	Low	High - website may go down
SQL Injection	Ensure that unapproved commands cannot be executed	Low	High - attackers may be able to access sensitive data

References

amosbastian, 2025. *amosbastian/understat: An asynchronous Python package for https://understat.com/*. [Online]
Available at: https://github.com/amosbastian/understat

API-Football, 2025. *Football - Restful API for Football data*. [Online]
Available at: https://www.api-football.com/

Chowdhury, T., 2025. *Premier League: Why is there a 3pm TV blackout in UK?*. [Online]
Available at: https://www.bbc.co.uk/sport/football/articles/c98l671604vo

FotMob, n.d. *FotMob*. [Online]
Available at: https://www.fotmob.com/

Jaja, D., 2025. *Building a real-time Livescore app with a Football API: Best practices*. [Online]
Available at: https://www.sportmonks.com/blogs/building-a-real-time-livescore-app-with-a-football-api-best-practices/

Statsbomb, n.d. *statsbomb/open-data: Free football data from StatsBomb*. [Online]
Available at: https://github.com/statsbomb/open-data